شرح سطر بسطر ١٤ — حسابي (19) ورمز الطالب (29)

مشروع رِزمة · Flutter · المرحلة 6 · كل سطر مع سبب كتابته بهذه الطريقة

شاشتان: حسابي (أربع مجموعات: الملف، الدراسة، التنزيل والتخزين، عام) ورمز الطالب (الرمز مع نسخ ومشاركة، وسبب وجوده، وحالة الحفظ على الإنترنت). الودجتات المشتركة والأوراق والتفضيلات في الملف التالي (١٥).

كل رقم ظاهر حقيقي: النجوم من مجموع `starsEarned` في المحاولات، والرمز السري «مفعّل» إن وُجدت تجزئته في الملف، وعدد النتائج المنتظرة من عدد صفوف طابور المزامنة. لا رقم منسوخ من Figma.

قرار مقصود: «الوضع الليلي» معطّل مع كلمة «قريبًا» لأن الثيم الداكن غير مبني ومفتاح لا يفعل شيئًا يخدع الطالب. ومفاتيح «أصوات الألعاب» و«Wi-Fi فقط» و«تنزيل الفيديو» تُحفظ الآن وتُقرأ حين تُبنى ميزاتها؛ وتقول الواجهة ذلك في ملاحظة «يعمل مع التنزيلات عند توفرها».

لا نتحقق من الرمز الحالي قبل تغييره أو إلغائه من حسابي، لأن الطالب أصلًا دخل ملفه بعد إدخاله (أو لأنه بلا رمز). هذا قرار مقصود ومذكور في ملف الشرح الموضوعي.

lib/app/tab_navigation.dart — ماذا يحدث عند ضغط تبويب

lib/app/tab_navigation.dart

الأسطر 1–2

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material (لـ `BuildContext`) و`go_router`.

لماذا بهذه الطريقة؟ الدالة تنتقل بـ `context.go`.

الأسطر 4–5

```
import '../core/widgets/bottom_nav.dart';
import 'routes.dart';
```

ماذا يفعل؟ استيراد الشريط السفلي (لتعداد `RizmaTab`) والمسارات.

لماذا بهذه الطريقة؟ التبويبات معرّفة في الشريط؛ المسارات في `routes.dart`.

الأسطر 7–8

```
/// ما يحدث عند ضغط تبويب في الشريط السفلي.
void onTabSelected(BuildContext context, RizmaTab current, RizmaTab target) {
```

ماذا يفعل؟ تعليق ودالة حرة `onTabSelected(context, current, target)`.

لماذا بهذه الطريقة؟ كل شاشة فيها الشريط السفلي تستدعي الدالة نفسها، فلا يتكرّر منطق التنقّل في أربع شاشات.

السطر 9

```
9     if (target == current) return;
```

ماذا يفعل؟ إن كان التبويب المختار هو الحالي لا نفعل شيئًا.

لماذا بهذه الطريقة؟ منع إعادة بناء الشاشة نفسها عند لمس تبويبها.

الأسطر 10–20

```
10    switch (target) {
11      case RizmaTab.home:
12        context.go(AppRoutes.home);
13      case RizmaTab.profile:
14        context.go(AppRoutes.account);
15      case RizmaTab.subjects:
16        context.go(AppRoutes.subjects);
17      case RizmaTab.progress:
18        context.go(AppRoutes.progress);
19    }
20  }
```

ماذا يفعل؟ `switch` على الهدف: الرئيسية، حسابي، المواد، تقدّمي، كلٌّ بـ `context.go` إلى مساره، ثم إغلاق الدالة.

لماذا بهذه الطريقة؟ `go` (لا `push`) لأن التبويبات أماكن متوازية، لا تراكم شاشات فوق بعضها. صارت الشاشات الأربع مبنية، فلم تعد هناك رسالة «قيد الإعداد».

lib/features/account/account_providers.dart — مزوّدات حسابي ورمز الطالب

`lib/features/account/account_providers.dart`

الأسطر 1–2

```
1   import 'package:flutter_riverpod/flutter_riverpod.dart';
2   import 'package:share_plus/share_plus.dart';
```

ماذا يفعل؟ استيراد Riverpod وحزمة `share_plus`.

لماذا بهذه الطريقة؟ المزوّدات من Riverpod، والمشاركة من `share_plus`.

الأسطر 4–6

```
4   import '../../core/db/database_provider.dart';
5   import '../profiles/active_profile_provider.dart';
6   import 'preferences_repository.dart';
```

ماذا يفعل؟ استيراد مزوّد القاعدة ومزوّد الملف النشط ومستودع التفضيلات.

لماذا بهذه الطريقة؟ كل المزوّدات هنا مبنية على هذه الثلاثة.

الأسطر 8–10

```
final preferencesRepositoryProvider = Provider<PreferencesRepository>((ref) {
  return PreferencesRepository(ref.watch(appDatabaseProvider));
});
```

ماذا يفعل؟ `preferencesRepositoryProvider`: ينشئ `PreferencesRepository` على القاعدة.

لماذا بهذه الطريقة؟ مستودع واحد للتطبيق كله، ويُستبدل في الاختبارات بسهولة.

الأسطر 12–20

```
/// تفضيلات الملف النشط (تتحدّث فور تغييرها). بلا ملف نشط: الافتراضيات.
final activePrefsProvider = StreamProvider<ProfilePrefs>((ref) async* {
  final id = await ref.watch(activeProfileIdProvider.future);
  if (id == null) {
    yield const ProfilePrefs();
    return;
  }
  yield* ref.watch(preferencesRepositoryProvider).watch(id);
});
```

ماذا يفعل؟ `activePrefsProvider`: بث بتفضيلات الملف النشط؛ بلا ملف نشط يبثّ الافتراضيات مرة ويتوقف.

لماذا بهذه الطريقة؟ `yield*` يمرّر بث القاعدة كما هو، فأي تغيير في الإعدادات يظهر فورًا في الشاشات.

الأسطر 22–30

```
/// الدروس المحفوظة للملف النشط.
final bookmarksProvider = StreamProvider<Set<String>>((ref) async* {
  final id = await ref.watch(activeProfileIdProvider.future);
  if (id == null) {
    yield const <String>{};
    return;
  }
  yield* ref.watch(preferencesRepositoryProvider).watchBookmarks(id);
});
```

ماذا يفعل؟ `bookmarksProvider`: بث بمعرّفات الدروس المحفوظة للملف النشط، أو مجموعة فارغة بلا ملف.

لماذا بهذه الطريقة؟ زر «حفظ الدرس» وقائمة المحفوظات يقرآن المصدر نفسه فيبقيان متطابقين.

الأسطر 32–35

```
/// عدد النتائج المحفوظة على الجهاز وتنتظر الإرسال (طابور المزامنة).
final pendingSyncCountProvider = StreamProvider.autoDispose<int>((ref) {
  return ref.watch(appDatabaseProvider).activityDao.watchPendingSyncCount();
});
```

ماذا يفعل؟ `pendingSyncCountProvider`: بث بعدد النتائج التي تنتظر الإرسال.

لماذا بهذه الطريقة؟ `autoDispose` لأنه لا يُحتاج إلا في شاشتي الرمز والتنزيلات.

الأسطر 37–38

```
/// يشارك نصًّا عبر نافذة المشاركة في النظام.
typedef ShareText = Future<void> Function(String text);
```

ماذا يفعل؟ `typedef ShareText` : دالة تأخذ نصًّا وتعيد `Future<void>` .

لماذا بهذه الطريقة؟ اسم واضح لنوع الدالة بدل كتابة التوقيع كاملًا في كل مكان.

الأسطر 40–45

```
/// مزوّد منفصل ليُستبدل في الاختبارات (نافذة النظام لا تعمل فيها).
final shareTextProvider = Provider<ShareText>((ref) {
  return (text) async {
    await SharePlus.instance.share(ShareParams(text: text));
  };
});
```

ماذا يفعل؟ `shareTextProvider` : يعيد دالة تفتح نافذة المشاركة في النظام بالنص.

لماذا بهذه الطريقة؟ مزوّد منفصل لأن نافذة النظام لا تعمل في الاختبارات؛ الاختبار يستبدله بدالة تسجّل النص فقط.

lib/features/account/pin_manage_sheet.dart — ورقة تغيير الرمز السري أو إلغائه

`lib/features/account/pin_manage_sheet.dart`

الأسطر 1–6

```
import 'package:flutter/material.dart';

import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/widgets/icon_tile.dart';
```

ماذا يفعل؟ استيراد Material والنصوص والألوان والأنماط ومربّع الأيقونة.

لماذا بهذه الطريقة؟ ما تحتاجه الورقة.

السطر 8

```
enum PinManageAction { change, disable }
```

ماذا يفعل؟ تعداد `PinManageAction { change, disable }` .

لماذا بهذه الطريقة؟ نعيد اختيار الطالب كقيمة مسمّاة لا كنص أو رقم، فيفحصها `switch` شاملة في الشاشة.

الأسطر 10–12

```
/// ورقة سفلية لمن لديه رمز سري مفعّل: تغييره أو إلغاؤه.
/// تعيد null إن أغلقها الطالب دون اختيار.
Future<PinManageAction?> showPinManageSheet(BuildContext context) {
```

ماذا يفعل؟ تعليق ودالة `showPinManageSheet` تعيد `Future<PinManageAction?>` .

لماذا بهذه الطريقة؟ `null` تعني أن الطالب أغلق الورقة دون اختيار؛ الشاشة تتجاهلها.

الأسطر 13–16

```
13    return showModalBottomSheet<PinManageAction>(
14      context: context,
15      shape: const RoundedRectangleBorder(
16        borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
```

ماذا يفعل؟ `showModalBottomSheet` بخلفية `ground` وزوايا علوية مدوّرة 28.

لماذا بهذه الطريقة؟ نفس شكل ورقتي الرمز السري و«نسيت الرمز؟» فتبدو كلها من عائلة واحدة.

الأسطر 17–19

```
17      ),
18      builder: (context) => SafeArea(
19        child: Padding(
```

ماذا يفعل؟ `SafeArea` ثم حشوة 20/24/20/16 (`Directional`).

لماذا بهذه الطريقة؟ تمنع تداخل الورقة مع حافة الجهاز السفلية.

الأسطر 20–24

```
20          padding: const EdgeInsetsDirectional.fromSTEB(20, 24, 20, 16),
21          child: Column(
22            mainAxisSize: MainAxisSize.min,
23            crossAxisAlignment: CrossAxisAlignment.stretch,
24            spacing: 8,
```

ماذا يفعل؟ `Column` بأقل حجم ممكن ومسافة 8 بين العناصر.

لماذا بهذه الطريقة؟ `mainAxisSize.min` فتأخذ الورقة طول محتواها فقط لا الشاشة كلها.

السطر 25

```
25            children: [
```

ماذا يفعل؟ عنوان الورقة.

لماذا بهذه الطريقة؟ بنمط `section18` كباقي العناوين.

الأسطر 26–32

```
26              Text(ArStrings.pinManageTitle, style: AppTextStyles.section18),
27              _ActionRow(
28                icon: Icons.edit_outlined,
29                iconColor: AppColors.teal,
30                tint: AppColors.tealTint,
31                label: ArStrings.pinChange,
32                onTap: () => Navigator.of(context).pop(PinManageAction.change),
```

ماذا يفعل؟ صف «تغيير الرمز»: أيقونة تعديل خضراء، وعند الضغط `pop(change)`.

لماذا بهذه الطريقة؟ `Navigator.pop` بقيمة تُعاد لمن فتح الورقة.

الأسطر 33–39

```
            ),
            _ActionRow(
              icon: Icons.lock_open_outlined,
              iconColor: AppColors.coral,
              tint: AppColors.coralTint,
              label: ArStrings.pinDisable,
              onTap: () => Navigator.of(context).pop(PinManageAction.disable),
```

ماذا يفعل؟ صف «إلغاء الرمز السري»: أيقونة قفل مفتوح مرجانية، وعند الضغط `pop(disable)`.

لماذا بهذه الطريقة؟ اللون المرجاني يلمّح أنه إجراء يُضعف الحماية.

الأسطر 40–46

```
            ),
          ],
        ),
      ),
    ),
  );
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 47–55

```
class _ActionRow extends StatelessWidget {
  const _ActionRow({
    required this.icon,
    required this.iconColor,
    required this.tint,
    required this.label,
    required this.onTap,
  });
```

ماذا يفعل؟ صنف خاص `_ActionRow` ومُنشئه بخمسة معاملات.

لماذا بهذه الطريقة؟ ودجت صغير خاص بهذا الملف لأن لا أحد غيره يحتاجه.

الأسطر 56–61

```
  final IconData icon;
  final Color iconColor;
  final Color tint;
  final String label;
  final VoidCallback onTap;
```

ماذا يفعل؟ الحقول: الأيقونة ولونها وخلفيتها والعنوان ودالة الضغط.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 62–64

```
62    @override
63    Widget build(BuildContext context) {
```

ماذا يفعل؟ `build` ويعيد `Material` ببطاقة بيضاء.

لماذا بهذه الطريقة؟ `Material` ضرورية ليظهر أثر اللمس (`InkWell`) فوق اللون.

الأسطر 65–67

```
65      return Material(
66        color: AppColors.card,
67        borderRadius: BorderRadius.circular(18),
```

ماذا يفعل؟ لون البطاقة وزوايا 18 و `clipBehavior` ليُقصّ أثر اللمس بالزوايا.

لماذا بهذه الطريقة؟ بدون القص يخرج الموج عن الزوايا المدوّرة.

الأسطر 68–72

```
68        clipBehavior: Clip.antiAlias,
69        child: InkWell(
70          onTap: onTap,
71          child: Padding(
72            padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
```

ماذا يفعل؟ `InkWell` للضغط، وحشوة أفقية 14 ورأسية 10، و `Row` بمسافة 12.

لماذا بهذه الطريقة؟ أول عنصر في `Row` على اليمين (RTL): مربّع الأيقونة.

الأسطر 73–74

```
73            child: Row(
74              spacing: 12,
```

ماذا يفعل؟ `spacing: 12` و `children` .

لماذا بهذه الطريقة؟ مسافة ثابتة بدل `SizedBox` بين كل عنصرين.

الأسطر 75–82

```
75              children: [
76                IconTile(
77                  iconData: icon,
78                  iconColor: iconColor,
79                  color: tint,
80                  size: 40,
81                  iconSize: 20,
82                  radius: 12,
```

ماذا يفعل؟ `IconTile` بحجم 40 وأيقونة 20 وزوايا 12.

لماذا بهذه الطريقة؟ نفس مقاسات صفوف الإعداد في التصميم.

السطر 83

```
83                ),
```

ماذا يفعل؟ `Expanded` فيه نص العنوان.

لماذا بهذه الطريقة؟ ليأخذ العرض المتبقي ولا يتجاوز الحافة إن طال.

الأسطر 84–91

```
84                Expanded(child: Text(label, style: AppTextStyles.body15)),
85              ],
86            ),
87          ),
88        ),
89      );
90    }
91  }
```

ماذا يفعل؟ إغلاق `Row` و `Padding` و `InkWell` و `Material` والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/features/account/account_screen.dart — الشاشة 19: حسابي (التصميم الجديد)

`lib/features/account/account_screen.dart`

المنطق (الرمز السري، تغيير الصف، حجم الخط، الحذف) كما كان. تغيّر الشكل حسب التصميم 38: عنوان بشارة «دون إنترنت»، بطاقة ملف بالتميمة في دائرة بحدّ بلون الطالب وشارة النجوم وزر «تعديل المظهر واللون»، وأيقونات الصفوف صارت من مجموعة أيقونات التصميم بدل أيقونات Material.

الأسطر 1–3

```
1   import 'package:flutter/material.dart';
2   import 'package:flutter_riverpod/flutter_riverpod.dart';
3   import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material وRiverpod و `go_router` .

لماذا بهذه الطريقة؟ الشاشة تقرأ الملف والتفضيلات من Riverpod وتنتقل بـ `go_router` .

الأسطر 5–30

```dart
import '../../app/routes.dart';
import '../../app/tab_navigation.dart';
import '../../core/db/app_database.dart';
import '../../core/strings/ar.dart';
import '../../core/strings/ar_progress.dart';
import '../../core/widgets/mascot_items.dart';
import '../wardrobe/wardrobe_providers.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/theme/profile_palette.dart';
import '../../core/widgets/bottom_nav.dart';
import '../../core/widgets/offline_chip.dart';
import '../../core/widgets/rizma_button.dart';
import '../../core/widgets/tag_chip.dart';
import '../../core/widgets/rizma_card.dart';
import '../catalog/catalog_providers.dart';
import '../profiles/active_profile_provider.dart';
import '../profiles/pin_setup_sheet.dart';
import '../profiles/profile_providers.dart';
import 'account_providers.dart';
import 'pin_manage_sheet.dart';
import 'preferences_repository.dart';
import 'widgets/account_sheets.dart';
import 'widgets/delete_profile_sheet.dart';
import 'widgets/setting_widgets.dart';
import '../../core/widgets/fade_slide_in.dart';
```

ماذا يفعل؟ استيراد المسارات وتنقّل التبويبات والقاعدة والنصوص والتميمة بملابسها ومزوّد الخزانة والألوان والودجات المشتركة والأوراق السفلية (الرمز، الصف، الحذف، المعلومات) وصفوف الإعدادات.

لماذا بهذه الطريقة؟ الشاشة تجمع أجزاء كثيرة؛ كل ورقة وكل صف في ملف خاص حتى يبقى هذا الملف عن الترتيب فقط.

الأسطر 32–36

```dart
/// الشاشة 19 «حسابي»: الملف، الدراسة، التنزيل والتخزين، عام.
/// «الوضع الليلي» معطّل («قريبًا») لأن الثيم الداكن لم يُبنَ بعد، ومفاتيح
/// التخزين والأصوات تُحفظ الآن وتُقرأ حين تُبنى ميزاتها.
class AccountScreen extends ConsumerWidget {
  const AccountScreen({super.key});
```

ماذا يفعل؟ تعليق: «الوضع الليلي» معطّل («قريبًا»)، ومفاتيح التخزين والأصوات تُحفظ الآن وتُقرأ حين تُبنى ميزاتها. ثم صنف `AccountScreen` من نوع `ConsumerWidget`.

لماذا بهذه الطريقة؟ نقول في الكود بصدق ما هو مبني وما ليس مبنيًّا بعد.

الأسطر 38–42

```dart
  void _note(BuildContext context, String message) {
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(message)));
  }
```

ماذا يفعل؟ `_note`: يعرض رسالة قصيرة أسفل الشاشة.

لماذا بهذه الطريقة؟ دالة واحدة لكل رسائل التأكيد.

الأسطر 44–51

```
  Future<void> _onPinTap(
    BuildContext context,
    WidgetRef ref,
    Profile profile,
  ) async {
    final repo = ref.read(profileRepositoryProvider);
    final enabled = profile.pinHash != null;
```

ماذا يفعل؟ `_onPinTap` : يقرأ المستودع ويعرف هل الرمز مفعّل.

لماذا بهذه الطريقة؟ السلوك يختلف: التفعيل لأول مرة أو إدارة رمز موجود.

الأسطر 52–58

```
    if (!enabled) {
      final pin = await showPinSetupSheet(context);
      if (pin == null) return;
      await repo.setPin(profile.id, pin);
      if (context.mounted) _note(context, ArStrings.pinEnabledNote);
      return;
    }
```

ماذا يفعل؟ غير مفعّل: ورقة إنشاء الرمز، ثم حفظه ورسالة «تم تفعيل الرمز».

لماذا بهذه الطريقة؟ إلغاء الورقة يعيد `null` فنخرج دون تغيير.

الأسطر 60–72

```
    final action = await showPinManageSheet(context);
    if (action == null || !context.mounted) return;
    switch (action) {
      case PinManageAction.change:
        final pin = await showPinSetupSheet(context);
        if (pin == null) return;
        await repo.setPin(profile.id, pin);
        if (context.mounted) _note(context, ArStrings.pinChangedNote);
      case PinManageAction.disable:
        await repo.clearPin(profile.id);
        if (context.mounted) _note(context, ArStrings.pinDisabledNote);
    }
  }
```

ماذا يفعل؟ مفعّل: ورقة «تغيير أو إيقاف»؛ التغيير يفتح ورقة إنشاء جديدة ويحفظ، والإيقاف يمسح الرمز؛ مع رسالة لكل حالة.

لماذا بهذه الطريقة؟ `switch` على التعداد يجبر المحلّل على تغطية الحالتين.

الأسطر 74–89

```dart
  Future<void> _onGradeTap(
    BuildContext context,
    WidgetRef ref,
    Profile profile,
  ) async {
    // صف الكتالوج المتوفّر؛ إن اختلف عن المختار نخبر الطالب.
    final contentGrade = ref.read(catalogProvider).value?.grade;
    final grade = await showGradeSheet(
      context,
      current: profile.grade,
      contentGrade: contentGrade,
    );
    if (grade == null || grade == profile.grade) return;
    await ref.read(profileRepositoryProvider).setGrade(profile.id, grade);
    if (context.mounted) _note(context, ArStrings.gradeChanged(grade));
  }
```

ماذا يفعل؟ `_onGradeTap` : يقرأ صف المحتوى المتوفر، يفتح ورقة الصفوف، ويحفظ الصف الجديد إن تغيّر مع رسالة.

لماذا بهذه الطريقة؟ الورقة تحذّر إن اختار الطالب صفًّا محتواه غير مثبّت بعد.

الأسطر 91–107

```dart
  Future<void> _onFontTap(
    BuildContext context,
    WidgetRef ref,
    Profile profile,
    ProfilePrefs prefs,
  ) async {
    final choice = await showChoiceSheet<FontSizeChoice>(
      context,
      title: ArStrings.fontSheetTitle,
      options: [for (final c in FontSizeChoice.values) (c, _fontSizeLabel(c))],
      selected: prefs.fontSize,
    );
    if (choice == null) return;
    await ref
        .read(preferencesRepositoryProvider)
        .setFontSize(profile.id, choice);
  }
```

ماذا يفعل؟ `_onFontTap` : ورقة خيارات بالأحجام الثلاثة، ويحفظ الاختيار.

لماذا بهذه الطريقة؟ `showChoiceSheet` عامة (`<T>`) وتُستعمل لأي اختيار من قائمة.

الأسطر 109–125

```
109   Future<void> _onDeleteTap(
110     BuildContext context,
111     WidgetRef ref,
112     Profile profile,
113   ) async {
114     final confirmed = await showDeleteProfileSheet(context, profile);
115     if (!confirmed || !context.mounted) return;
116     try {
117       await ref.read(profileRepositoryProvider).delete(profile.id);
118     } catch (_) {
119       if (context.mounted) _note(context, ArStrings.deleteFailed);
120       return;
121     }
122     await ref.read(activeProfileIdProvider.notifier).reload();
123     // شاشة الإقلاع تقرر الوجهة: اختيار ملف إن بقي أكثر من ملف، أو الترحيب إن لم يبقَ أحد.
124     if (context.mounted) context.go(AppRoutes.launch);
125   }
```

ماذا يفعل؟ `_onDeleteTap` : تأكيد الحذف، ثم الحذف (برسالة إن فشل)، ثم إعادة تحميل الملف النشط والعودة لشاشة الإقلاع.

لماذا بهذه الطريقة؟ شاشة الإقلاع تقرّر الوجهة: اختيار ملف إن بقي غيره، أو الترحيب إن لم يبقَ أحد.

الأسطر 127–132

```
127   @override
128   Widget build(BuildContext context, WidgetRef ref) {
129     final profile = ref.watch(activeProfileProvider);
130     final stars = ref.watch(starsByProfileProvider).value ?? const {};
131     final prefs = ref.watch(activePrefsProvider).value ?? const ProfilePrefs();
132     final prefsRepo = ref.read(preferencesRepositoryProvider);
```

ماذا يفعل؟ `build` : الملف النشط، ونجوم كل ملف، والتفضيلات (أو الافتراضيات)، ومستودع التفضيلات.

لماذا بهذه الطريقة؟ `read` للمستودع لأنه لا يتغيّر؛ `watch` لما يتغيّر.

الأسطر 134–152

```
    return Scaffold(
      body: SafeArea(
        bottom: false,
        child: profile.when(
          loading: () => const SizedBox.shrink(),
          error: (_, _) => Center(
            child: Text(
              ArStrings.accountLoadFailed,
              style: AppTextStyles.body15,
            ),
          ),
          data: (p) => p == null
              ? const SizedBox.shrink()
              : SingleChildScrollView(
                  padding: const EdgeInsetsDirectional.fromSTEB(20, 18, 20, 24),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.stretch,
                    spacing: 14,
                    children: [
```

ماذا يفعل؟ `Scaffold` و`SafeArea` (بلا حشوة سفلية لأن الشريط السفلي يحجزها)؛ تحميل لا شيء، خطأ رسالة، ثم تمرير وعمود بمسافة 14.

لماذا بهذه الطريقة؟ `when` يغطي الحالات الثلاث للبيانات بوضوح.

الأسطر 153–168

```
                      Row(
                        children: [
                          const Expanded(
                            child: Text(
                              ArStrings.accountTitle,
                              style: TextStyle(
                                fontFamily: AppTextStyles.fontFamily,
                                fontSize: 22,
                                fontWeight: FontWeight.w800,
                                color: AppColors.ink,
                              ),
                            ),
                          ),
                          const OfflineChip(),
                        ],
                      ).fadeIn(0),
```

ماذا يفعل؟ صف العنوان «حسابي» وشارة «دون إنترنت».

لماذا بهذه الطريقة؟ نفس رأس شاشات التبويبات الأخرى في التصميم.

الأسطر 169–174

```
                    _ProfileHeader(
                      profile: p,
                      stars: stars[p.id] ?? 0,
                      worn: ref.watch(activeEquippedProvider),
                      onEditLook: () => context.push(AppRoutes.wardrobe),
                    ).fadeIn(1),
```

ماذا يفعل؟ بطاقة الملف: الاسم والنجوم والملابس الملبوسة وزر «تعديل المظهر» إلى الخزانة.

لماذا بهذه الطريقة؟ التميمة تظهر بملابسها كما اختارها الطالب.

الأسطر 175–182

```
                    _ProfileGroup(
                      profile: p,
                      onStudentCode: () =>
                          context.push(AppRoutes.studentCode),
                      onSwitch: () => context.go(AppRoutes.profiles),
                      onWardrobe: () => context.push(AppRoutes.wardrobe),
                      onPin: () => _onPinTap(context, ref, p),
                    ).fadeIn(2),
```

ماذا يفعل؟ مجموعة «الملف»: رمز الطالب (`push`)، تبديل الطالب (`go`)، الخزانة (`push`)، الرمز السري.

لماذا بهذه الطريقة؟ `go` لتبديل الطالب لأنها بداية جلسة طالب آخر لا شاشة فرعية.

الأسطر 183–195

```
                    _StudyGroup(
                      profile: p,
                      prefs: prefs,
                      onGrade: () => _onGradeTap(context, ref, p),
                      onFontSize: () => _onFontTap(context, ref, p, prefs),
                      onGameSounds: (v) => prefsRepo.setGameSounds(p.id, v),
                    ).fadeIn(3),
                    _StorageGroup(
                      prefs: prefs,
                      onWifiOnly: (v) => prefsRepo.setWifiOnly(p.id, v),
                      onAutoVideo: (v) => prefsRepo.setAutoVideo(p.id, v),
                      onManage: () => context.push(AppRoutes.downloads),
                    ).fadeIn(4),
```

ماذا يفعل؟ مجموعة «الدراسة» (الصف، حجم الخط، أصوات الألعاب)، ومجموعة «التنزيل والتخزين» (Wi-Fi فقط، تشغيل الفيديو تلقائيًا، إدارة التنزيلات).

لماذا بهذه الطريقة؟ كل مفتاح يحفظ في القاعدة فورًا عبر المستودع.

الأسطر 196–215

```
                      _GeneralGroup(
                        onPrivacy: () => showInfoSheet(
                          context,
                          title: ArStrings.privacyTitle,
                          paragraphs: const [
                            ArStrings.privacy1,
                            ArStrings.privacy2,
                            ArStrings.privacy3,
                          ],
                        ),
                        onAbout: () => showInfoSheet(
                          context,
                          title: ArStrings.aboutTitle,
                          paragraphs: const [
                            ArStrings.about1,
                            ArStrings.about2,
                            ArStrings.about3,
                          ],
                        ),
                      ).fadeIn(5),
```

ماذا يفعل؟ مجموعة «عام»: ورقتا «الخصوصية» و«عن رِزمة» بثلاث فقرات لكلٍّ.

لماذا بهذه الطريقة؟ النصوص في ملف النصوص؛ الورقة عامة تعرض أي فقرات.

الأسطر 216–231

```
                      _DeleteCard(
                        onTap: () => _onDeleteTap(context, ref, p),
                      ).fadeIn(6),
                    ],
                  ),
                ),
          ),
        ),
        bottomNavigationBar: BottomNav(
          current: RizmaTab.profile,
          onSelected: (tab) => onTabSelected(context, RizmaTab.profile, tab),
        ),
      );
    }
}
```

ماذا يفعل؟ بطاقة حذف الملف، ثم الشريط السفلي على تبويب «حسابي»، وإغلاق الأقواس.

لماذا بهذه الطريقة؟ الحذف منفصل في الأسفل بلون مرجاني حتى لا يُلمس خطأً.

الأسطر 232–236

```
String _fontSizeLabel(FontSizeChoice choice) => switch (choice) {
  FontSizeChoice.small => ArStrings.fontSizeSmall,
  FontSizeChoice.medium => ArStrings.fontSizeMedium,
  FontSizeChoice.large => ArStrings.fontSizeLarge,
};
```

ماذا يفعل؟ `_fontSizeLabel`: اسم كل حجم خط بالعربية.

لماذا بهذه الطريقة؟ دالة واحدة تُستعمل في الورقة وفي قيمة الصف.

الأسطر 238–250

```
class _ProfileHeader extends StatelessWidget {
  const _ProfileHeader({
    required this.profile,
    required this.stars,
    required this.worn,
    required this.onEditLook,
  });

  final Profile profile;
  final int stars;
  final Set<String> worn;
  final VoidCallback onEditLook;
```

ماذا يفعل؟ `_ProfileHeader`: الملف والنجوم والملابس ودالة «تعديل المظهر».

لماذا بهذه الطريقة؟ الودجت لا يعرف التنقّل؛ يستقبل دالة جاهزة.

الأسطر 251–258

```
  @override
  Widget build(BuildContext context) {
    // بطاقة الملف (التصميم 38): التميمة في دائرة بجدّ بلون الملف، الاسم
    // شارة الصف والنجوم، وزر «تعديل المظهر واللون» إلى الخزانة.
    return RizmaCard(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 20),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
```

ماذا يفعل؟ تعليق، ثم `RizmaCard` بحشوة 16/20 وعمود ممتدّ بمسافة 6.

لماذا بهذه الطريقة؟ بطاقة بيضاء كما في التصميم 38.

الأسطر 259–277

```
        spacing: 6,
        children: [
          Center(
            child: Container(
              width: 112,
              height: 112,
              margin: const EdgeInsets.only(bottom: 6),
              decoration: BoxDecoration(
                color: profile.color.avatarBackground,
                shape: BoxShape.circle,
                border: Border.all(color: profile.color.body, width: 3),
              ),
              alignment: Alignment.center,
              child: MascotWithItems(
                size: 84,
                color: profile.color,
                items: worn,
              ),
            ),
```

ماذا يفعل؟ دائرة 112 بخلفية لون الطالب الفاتح وحدّ بلونه الأساسي (3)، وفيها التميمة 84 بملابسها.

لماذا بهذه الطريقة؟ الحدّ الملوّن من التصميم الجديد؛ يربط البطاقة بلون الطالب.

الأسطر 278–291

```
          ),
          Text(
            profile.name,
            textAlign: TextAlign.center,
            style: AppTextStyles.title26,
          ),
          Center(
            child: TagChip(
              label: ArStrings.accountSummary(profile.grade, stars),
              icon: 'starf',
              background: AppColors.amberTint,
              foreground: AppColors.amberInk,
            ),
          ),
```

ماذا يفعل؟ الاسم بخط 26 في المنتصف، ثم شارة عنبرية «الصف · النجوم» بنجمة.

لماذا بهذه الطريقة؟ الشارة تجمع معلومتين في سطر واحد قصير.

الأسطر 292–304

```
          const SizedBox(height: 6),
          RizmaButton(
            label: ArStrings.accountEditLook,
            kind: ButtonKind.soft,
            icon: 'starf',
            height: 46,
            onPressed: onEditLook,
          ),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ زر ناعم «تعديل المظهر واللون» يفتح الخزانة.

لماذا بهذه الطريقة؟ طريق مباشر لتزيين التميمة، أكثر ما يحبّه الأطفال في حسابهم.

الأسطر 306–320

```
class _ProfileGroup extends StatelessWidget {
  const _ProfileGroup({
    required this.profile,
    required this.onStudentCode,
    required this.onSwitch,
    required this.onWardrobe,
    required this.onPin,
  });

  final Profile profile;
  final VoidCallback onStudentCode;
  final VoidCallback onSwitch;
  final VoidCallback onWardrobe;
  final VoidCallback onPin;
```

ماذا يفعل؟ `_ProfileGroup` : الملف وأربع دوال.

لماذا بهذه الطريقة؟ الودجت قابل للاختبار لأنه لا يعرف التنقّل.

الأسطر 321–337

```
  @override
  Widget build(BuildContext context) {
    final pinOn = profile.pinHash != null;

    return SettingsGroupCard(
      title: ArStrings.accountProfileGroup,
      children: [
        SettingRow(
          icon: 'key',
          iconColor: AppColors.teal,
          tint: AppColors.tealTint,
          label: ArStrings.accountStudentCode,
          value: profile.studentCode,
          valueColor: AppColors.teal,
          valueLtr: true,
          onTap: onStudentCode,
        ),
```

ماذا يفعل؟ `pinOn` ، ثم مجموعة «الملف»: صف «رمز الطالب» بأيقونة مفتاح والرمز قيمةً خضراء من اليسار لليمين.

لماذا بهذه الطريقة؟ `valueLtr` حتى لا ينقلب الرمز في الواجهة العربية.

الأسطر 338–351

```
        SettingRow(
          icon: 'users',
          iconColor: AppColors.blue,
          tint: AppColors.blueTint,
          label: ArStrings.accountSwitch,
          onTap: onSwitch,
        ),
        SettingRow(
          icon: 'starf',
          iconColor: AppColors.amberDark,
          tint: AppColors.amberTint,
          label: ArProgress.wardrobeOpen,
          onTap: onWardrobe,
        ),
```

ماذا يفعل؟ صف «تبديل الطالب» (أيقونة مستخدمين زرقاء)، وصف «خزانة رِزمة» (نجمة عنبرية).

لماذا بهذه الطريقة؟ أيقونات من مجموعة التصميم بلون وخلفية مختلفين لكل صف.

الأسطر 352–364

```
        SettingRow(
          icon: 'lock',
          iconColor: AppColors.purple,
          tint: AppColors.purpleTint,
          label: ArStrings.accountPin,
          value: pinOn ? ArStrings.accountPinOn : ArStrings.accountPinOff,
          valueColor: pinOn ? AppColors.teal : AppColors.soft,
          onTap: onPin,
        ),
      ],
    );
  }
}
```

ماذا يفعل؟ صف «الرمز السري» (قفل بنفسجي) والقيمة «مفعّل» خضراء أو «غير مفعّل» رمادية، ثم الإغلاق.

لماذا بهذه الطريقة؟ اللون إشارة سريعة للحالة.

الأسطر 366–380

```
class _StudyGroup extends StatelessWidget {
  const _StudyGroup({
    required this.profile,
    required this.prefs,
    required this.onGrade,
    required this.onFontSize,
    required this.onGameSounds,
  });

  final Profile profile;
  final ProfilePrefs prefs;
  final VoidCallback onGrade;
  final VoidCallback onFontSize;
  final ValueChanged<bool> onGameSounds;
```

ماذا يفعل؟ `_StudyGroup` : الملف والتفضيلات وثلاث دوال.

لماذا بهذه الطريقة؟ مجموعة مستقلة لكل قسم من التصميم.

الأسطر 381–413

```
  @override
  Widget build(BuildContext context) {
    return SettingsGroupCard(
      title: ArStrings.accountStudyGroup,
      children: [
        SettingRow(
          icon: 'book',
          iconColor: AppColors.teal,
          tint: AppColors.tealTint,
          label: ArStrings.accountGrade,
          value: ArStrings.gradeOrdinal(profile.grade),
          onTap: onGrade,
        ),
        SettingRow(
          icon: 'type',
          iconColor: AppColors.blue,
          tint: AppColors.blueTint,
          label: ArStrings.accountFontSize,
          value: _fontSizeLabel(prefs.fontSize),
          onTap: onFontSize,
        ),
        SettingSwitchRow(
          icon: 'volume',
          iconColor: AppColors.purple,
          tint: AppColors.purpleTint,
          label: ArStrings.accountGameSounds,
          value: prefs.gameSounds,
          onChanged: onGameSounds,
        ),
      ],
    );
  }
}
```

ماذا يفعل؟ صف «الصف» (كتاب) بقيمة «الخامس»، صف «حجم الخط» (حرف) بقيمته، ومفتاح «أصوات الألعاب» (مكبّر صوت).

لماذا بهذه الطريقة؟ القيم تُقرأ من التفضيلات فتتحدّث فور التغيير.

الأسطر 415–427

```
class _StorageGroup extends StatelessWidget {
  const _StorageGroup({
    required this.prefs,
    required this.onWifiOnly,
    required this.onAutoVideo,
    required this.onManage,
  });

  final ProfilePrefs prefs;
  final ValueChanged<bool> onWifiOnly;
  final ValueChanged<bool> onAutoVideo;
  final VoidCallback onManage;
```

ماذا يفعل؟ `_StorageGroup` : التفضيلات ودوال المفتاحين وإدارة التنزيلات.

لماذا بهذه الطريقة؟ قسم «التنزيل والتخزين».

الأسطر 428–460

```
  @override
  Widget build(BuildContext context) {
    return SettingsGroupCard(
      title: ArStrings.accountStorageGroup,
      footnote: ArStrings.accountStorageNote,
      children: [
        SettingSwitchRow(
          icon: 'wifi',
          iconColor: AppColors.teal,
          tint: AppColors.tealTint,
          label: ArStrings.accountWifiOnly,
          value: prefs.wifiOnly,
          onChanged: onWifiOnly,
        ),
        SettingSwitchRow(
          icon: 'video',
          iconColor: AppColors.coral,
          tint: AppColors.coralTint,
          label: ArStrings.accountAutoVideo,
          value: prefs.autoVideo,
          onChanged: onAutoVideo,
        ),
        SettingRow(
          icon: 'download',
          iconColor: AppColors.blue,
          tint: AppColors.blueTint,
          label: ArStrings.accountManageDownloads,
          onTap: onManage,
        ),
      ],
    );
  }
}
```

ماذا يفعل؟ ملاحظة أسفل المجموعة، ومفتاح «Wi-Fi فقط» (wifi)، ومفتاح «تشغيل الفيديو تلقائيًا» (فيديو مرجاني)، وصف «إدارة التنزيلات» (تنزيل أزرق).

لماذا بهذه الطريقة؟ المفتاحان يُحفظان الآن؛ الملاحظة تقول إن أثرهما يأتي مع ميزة التنزيل.

الأسطر 462–481

```
462 class _DeleteCard extends StatelessWidget {
463   const _DeleteCard({required this.onTap});
464
465   final VoidCallback onTap;
466
467   @override
468   Widget build(BuildContext context) {
469     return RizmaCard(
470       padding: const EdgeInsetsDirectional.fromSTEB(16, 4, 16, 4),
471       child: SettingRow(
472         icon: 'trash',
473         iconColor: AppColors.coral,
474         tint: AppColors.coralTint,
475         label: ArStrings.accountDelete,
476         labelColor: AppColors.coral,
477         onTap: onTap,
478       ),
479     );
480   }
481 }
```

ماذا يفعل؟ `_DeleteCard` : بطاقة فيها صف «حذف هذا الملف» بأيقونة سلة ولون مرجاني للنص.

لماذا بهذه الطريقة؟ بطاقة منفصلة تفصل الإجراء الخطِر عن الإعدادات العادية.

الأسطر 483–517

```
class _GeneralGroup extends StatelessWidget {
  const _GeneralGroup({required this.onPrivacy, required this.onAbout});

  final VoidCallback onPrivacy;
  final VoidCallback onAbout;

  @override
  Widget build(BuildContext context) {
    return SettingsGroupCard(
      title: ArStrings.accountGeneralGroup,
      children: [
        const SoonSettingRow(
          icon: 'moon',
          iconColor: AppColors.soft,
          tint: AppColors.mute,
          label: ArStrings.accountNightMode,
        ),
        SettingRow(
          icon: 'shield',
          iconColor: AppColors.teal,
          tint: AppColors.tealTint,
          label: ArStrings.accountPrivacy,
          onTap: onPrivacy,
        ),
        SettingRow(
          icon: 'globe',
          iconColor: AppColors.amberDark,
          tint: AppColors.amberTint,
          label: ArStrings.accountAbout,
          onTap: onAbout,
        ),
      ],
    );
  }
}
```

ماذا يفعل؟ `_GeneralGroup` : «الوضع الليلي» معطّل بشارة «قريبًا» (قمر)، و«الخصوصية» (درع)، و«عن رِزمة» (كرة أرضية).

لماذا بهذه الطريقة؟ صف «قريبًا» يعرض الميزة القادمة بصدق دون أن يوهم بأنها تعمل.

lib/features/account/student_code_screen.dart — الشاشة 29: رمز الطالب (التصميم الجديد)

`lib/features/account/student_code_screen.dart`

أُعيد بناء الشاشة حسب التصميم 06: رأس «بطاقة الطالب» بالتميمة، بطاقة الرمز بشريط أخضر و«صالح دائمًا»، تنبيه عنبري، زرّا النسخ والمشاركة، ثم «لماذا هذا الرمز؟» وحالة الحفظ ومجموعة الصف.

الأسطر 1–4

```dart
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material و `services` (للحافظة) وRiverpod و `go_router` .

لماذا بهذه الطريقة؟ `Clipboard` في `services` ؛ الباقي للودجات والحالة والتنقّل.

الأسطر 6–20

```dart
import '../../app/routes.dart';
import '../../core/db/app_database.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/widgets/app_icon.dart';
import '../../core/widgets/fade_slide_in.dart';
import '../../core/widgets/icon_tile.dart';
import '../../core/widgets/ledge.dart';
import '../../core/widgets/mascot.dart';
import '../../core/widgets/rizma_button.dart';
import '../../core/widgets/screen_top_bar.dart';
import '../../core/widgets/tag_chip.dart';
import '../profiles/active_profile_provider.dart';
import 'account_providers.dart';
```

ماذا يفعل؟ استيراد المسارات والقاعدة والنصوص والألوان والخطوط والودجات المشتركة، ومزوّد الملف النشط ومزوّدات الحساب.

لماذا بهذه الطريقة؟ الشاشة تجمع مكوّنات جاهزة فلا تحتوي إلا ترتيبها.

الأسطر 22–26

```dart
/// الشاشة 29 «رمز الطالب» (التصميم الجديد 06): بطاقة الطالب، بطاقة الرمز
/// بشريطها الأخضر، تنبيه «احفظه في مكان آمن»، زرّا النسخ والمشاركة، ثم
/// سبب الرمز وحالة الحفظ على الإنترنت ومجموعة الصف إن وُجدت.
class StudentCodeScreen extends ConsumerWidget {
  const StudentCodeScreen({super.key});
```

ماذا يفعل؟ تعليق يلخّص أقسام الشاشة، وصنف `StudentCodeScreen` من نوع `ConsumerWidget` .

لماذا بهذه الطريقة؟ كل البيانات من المزوّدات؛ لا حالة محلية.

الأسطر 28–30

```dart
  void _back(BuildContext context) {
    context.canPop() ? context.pop() : context.go(AppRoutes.account);
  }
```

ماذا يفعل؟ `_back` : رجوع للخلف إن أمكن، وإلا إلى «حسابي».

لماذا بهذه الطريقة؟ يحمي حالة فتح الشاشة مباشرة دون شاشة قبلها.

الأسطر 32–38

```
32    Future<void> _copy(BuildContext context, String code) async {
33      await Clipboard.setData(ClipboardData(text: code));
34      if (!context.mounted) return;
35      ScaffoldMmessenger.of(
36        context,
37      ).showSnackBar(const SnackBar(content: Text(ArStrings.codeCopied)));
38    }
```

ماذا يفعل؟ `_copy` : ينسخ الرمز إلى الحافظة ثم يعرض «تم نسخ الرمز».

لماذا بهذه الطريقة؟ تأكيد مرئي لأن النسخ لا يظهر أثره وحده. فحص `mounted` بعد `await` .

الأسطر 40–44

```
40    @override
41    Widget build(BuildContext context, WidgetRef ref) {
42      final profile = ref.watch(activeProfileProvider).value;
43      final pending = ref.watch(pendingSyncCountProvider).value ?? 0;
```

ماذا يفعل؟ `build` : الملف النشط وعدد النتائج المنتظرة للمزامنة.

لماذا بهذه الطريقة؟ العدد يظهر في سطر «حالة الحفظ» أسفل الشاشة.

الأسطر 45–56

```
45      return Scaffold(
46        body: SafeArea(
47          child: profile == null
48              ? const SizedBox.shrink()
49              : SingleChildScrollView(
50                  child: Column(
51                    crossAxisAlignment: CrossAxisAlignment.stretch,
52                    children: [
53                      ScreenTopBar(
54                        title: ArStrings.codeTitle,
55                        onBack: () => _back(context),
56                      ),
```

ماذا يفعل؟ `Scaffold` و `SafeArea` ؛ قبل وصول الملف لا شيء، ثم تمرير وعمود أوله الشريط العلوي «رمز الطالب».

لماذا بهذه الطريقة؟ الشريط جزء من التمرير فلا يأخذ مساحة ثابتة على الشاشات الصغيرة.

الأسطر 57–67

```
                    Padding(
                      padding: const EdgeInsetsDirectional.fromSTEB(
                        20,
                        8,
                        20,
                        24,
                      ),
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.stretch,
                        spacing: 14,
                        children: [
```

ماذا يفعل؟ حشوة وعمود بمسافة 14.

لماذا بهذه الطريقة؟ مسافة موحّدة بين البطاقات كما في التصميم.

الأسطر 68–70

```
                          _StudentHeader(profile: profile).fadeIn(0),
                          _CodeCard(code: profile.studentCode).fadeIn(1),
                          const _KeepSafeCard().fadeIn(2),
```

ماذا يفعل؟ رأس الطالب، ثم بطاقة الرمز، ثم التنبيه العنبري، بظهور متدرّج.

لماذا بهذه الطريقة؟ الترتيب من الأهم: من صاحب الرمز، ثم الرمز، ثم كيف يحفظه.

الأسطر 71–84

```
                          RizmaButton(
                            label: ArStrings.codeCopy,
                            icon: 'copy',
                            onPressed: () =>
                                _copy(context, profile.studentCode),
                          ),
                          RizmaButton(
                            label: ArStrings.codeShare,
                            icon: 'share',
                            kind: ButtonKind.outlined,
                            onPressed: () => ref.read(shareTextProvider)(
                              ArStrings.codeShareMessage(profile.studentCode),
                            ),
                          ),
```

ماذا يفعل؟ زر «نسخ الرمز» الممتلئ، وزر «مشاركة الرمز» بحدّ فقط يرسل رسالة جاهزة فيها الرمز.

لماذا بهذه الطريقة؟ زرّان متتاليان كما في التصميم؛ المشاركة عبر المزوّد ليُستبدل في الاختبارات.

الأسطر 85–94

```
                          const _WhyCard().fadeIn(3),
                          _InfoRow(
                            icon: 'sync',
                            iconColor: AppColors.teal,
                            tint: AppColors.tealTint,
                            title: ArStrings.codeNeverSynced,
                            caption: pending == 0
                                ? ArStrings.codeNothingPending
                                : ArStrings.codePending(pending),
                          ),
```

ماذا يفعل؟ بطاقة «لماذا هذا الرمز؟»، ثم سطر حالة الحفظ: «لم يُحفظ على الإنترنت بعد» مع عدد النتائج المنتظرة أو «لا شيء ينتظر».

لماذا بهذه الطريقة؟ صدق مع الطالب: المزامنة غير مبنية بعد، فنقول إن النتائج محفوظة على الجهاز وتنتظر.

الأسطر 95–114

```
                          if (profile.classCode != null)
                            _InfoRow(
                              icon: 'users',
                              iconColor: AppColors.amberDark,
                              tint: AppColors.amberTint,
                              title: ArStrings.codeClassGroup,
                              caption: ArStrings.codeClassLine(
                                profile.classCode!,
                              ),
                            ),
                        ],
                      ),
                    ),
                  ],
                ),
              ),
            ),
        ),
      );
  }
}
```

ماذا يفعل؟ إن كان للطالب رمز صف: سطر «مجموعة الصف» برمزه. ثم إغلاق الأقواس.

لماذا بهذه الطريقة؟ يظهر فقط حين يوجد معنى له.

الأسطر 116–121

```
/// بطاقة الطالب: {الاسم} بجانب التميمة، وشارة الصف».
class _StudentHeader extends StatelessWidget {
  const _StudentHeader({required this.profile});

  final Profile profile;
```

ماذا يفعل؟ `_StudentHeader`: يستقبل الملف.

لماذا بهذه الطريقة؟ رأس مستقل يُقرأ بسهولة.

الأسطر 122–147

```
  @override
  Widget build(BuildContext context) {
    return Row(
      spacing: 12,
      children: [
        Expanded(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            spacing: 4,
            children: [
              Text(
                ArStrings.codeStudentCard(profile.name),
                style: AppTextStyles.heading22,
              ),
              TagChip(
                label: ArStrings.gradeName(profile.grade),
                icon: 'book',
              ),
            ],
          ),
        ),
        Mascot(size: 64, color: profile.color),
      ],
    );
  }
}
```

ماذا يفعل؟ صف: «بطاقة الطالب: الاسم» وتحته شارة الصف، وبجانبه التميمة 64 بلون الطالب.

لماذا بهذه الطريقة؟ `Expanded` للنص حتى يلتف الاسم الطويل دون أن يدفع التميمة خارج الشاشة.

الأسطر 149–155

```
/// بطاقة الرمز: شريط أخضر «رمز الاستعادة والربط» و«صالح دائمًا»، ثم الرمز
/// كبيرًا داخل صندوق سماوي.
class _CodeCard extends StatelessWidget {
  const _CodeCard({required this.code});

  final String code;
```

ماذا يفعل؟ تعليق وصنف `_CodeCard` بالرمز.

لماذا بهذه الطريقة؟ البطاقة الأهم في الشاشة.

الأسطر 156–165

```
156   @override
157   Widget build(BuildContext context) {
158     return LedgeBox(
159       radius: 22,
160       edge: AppColors.tealDark,
161       border: Border.all(color: AppColors.teal, width: 2),
162       clip: true,
163       child: Column(
164         crossAxisAlignment: CrossAxisAlignment.stretch,
165         children: [
```

ماذا يفعل؟ `LedgeBox` بزوايا 22 وحدّ أخضر وحافة خضراء داكنة وقصّ، وعمود ممتدّ.

لماذا بهذه الطريقة؟ الحدّ الأخضر يميّز بطاقة الرمز عن البطاقات البيضاء الأخرى. القصّ يُبقي الشريط داخل الزوايا.

الأسطر 166–188

```
166           Container(
167             color: AppColors.teal,
168             padding: const EdgeInsetsDirectional.fromSTEB(14, 10, 14, 10),
169             child: Row(
170               spacing: 8,
171               children: [
172                 const AppIcon('key', size: 18, color: AppColors.card),
173                 Expanded(
174                   child: Text(
175                     ArStrings.codeBandTitle,
176                     style: AppTextStyles.label14.copyWith(
177                       color: AppColors.card,
178                     ),
179                   ),
180                 ),
181                 const TagChip(
182                   label: ArStrings.codeAlwaysValid,
183                   background: AppColors.mint,
184                   foreground: AppColors.tealDark,
185                 ),
186               ],
187             ),
188           ),
```

ماذا يفعل؟ الشريط الأخضر: مفتاح، «رمز الاستعادة والربط»، وشارة نعناعية «صالح دائمًا».

لماذا بهذه الطريقة؟ يشرح وظيفة الرمز في سطر، ويطمئن أنه لا ينتهي.

الأسطر 189–194

```
189           Padding(
190             padding: const EdgeInsets.all(16),
191             child: Column(
192               spacing: 8,
193               children: [
194                 Text(ArStrings.codeYours, style: AppTextStyles.caption13),
```

ماذا يفعل؟ حشوة 16 وعمود: سطر «رمزك» الصغير.

لماذا بهذه الطريقة؟ عنوان صغير فوق الرمز.

الأسطر 195–220

```
                Container(
                  width: double.infinity,
                  padding: const EdgeInsets.symmetric(vertical: 12),
                  decoration: BoxDecoration(
                    color: AppColors.sky,
                    borderRadius: BorderRadius.circular(16),
                  ),
                  alignment: Alignment.center,
                  child: Text(
                    code,
                    textDirection: TextDirection.ltr,
                    style: AppTextStyles.display30.copyWith(
                      color: AppColors.teal,
                      fontSize: 34,
                      letterSpacing: 1.5,
                    ),
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ صندوق سماوي بعرض كامل والرمز في وسطه بخط 34 أخضر واتجاه من اليسار لليمين وتباعد أحرف.

لماذا بهذه الطريقة؟ `ltr` حتى لا ينقلب «RZ-7K4M» في واجهة عربية؛ التباعد يسهّل نقل الرمز حرفًا حرفًا.

الأسطر 222–227

```
/// تنبيه عنبري: «احفظ هذا الرمز في مكان آمن».
class _KeepSafeCard extends StatelessWidget {
  const _KeepSafeCard();

  @override
  Widget build(BuildContext context) {
```

ماذا يفعل؟ `_KeepSafeCard` : تنبيه عنبري.

لماذا بهذه الطريقة؟ صنف مستقل لأنه كتلة واضحة في التصميم.

الأسطر 228–266

```
    return LedgeBox(
      color: AppColors.amberTint,
      radius: 18,
      padding: const EdgeInsets.all(14),
      child: Row(
        spacing: 12,
        children: [
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  ArStrings.codeKeepSafeTitle,
                  style: AppTextStyles.h3_17.copyWith(
                    color: AppColors.amberInk,
                  ),
                ),
                Text(
                  ArStrings.codeKeepSafe,
                  style: AppTextStyles.caption13.copyWith(
                    color: AppColors.amberInk,
                  ),
                ),
              ],
            ),
          ),
          const IconTile(
            iconName: 'warn',
            iconColor: AppColors.amberDark,
            color: AppColors.card,
            size: 40,
            iconSize: 20,
            radius: 12,
          ),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ `LedgeBox` عنبري: عنوان «احفظه في مكان آمن» وسطر شرح بلون حبر عنبري، ومربّع أيقونة التحذير.

لماذا بهذه الطريقة؟ العنبري للتنبيه لا للخطأ؛ الطفل يُطلب منه أن يكتب الرمز أو يعطيه لوليّ أمره.

الأسطر 268–289

```dart
/// «لماذا هذا الرمز؟» بثلاثة أسباب.
class _WhyCard extends StatelessWidget {
  const _WhyCard();

  @override
  Widget build(BuildContext context) {
    return LedgeBox(
      radius: 20,
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        spacing: 2,
        children: [
          Text(ArStrings.codeWhyTitle, style: AppTextStyles.h3_17),
          const _WhyRow(icon: 'phone', text: ArStrings.codeWhy1),
          const _WhyRow(icon: 'users', text: ArStrings.codeWhy2),
          const _WhyRow(icon: 'shield', text: ArStrings.codeWhy3),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ `_WhyCard`: بطاقة «لماذا هذا الرمز؟» وثلاثة أسطر: جهاز آخر، المعلّم، الأمان.

لماذا بهذه الطريقة؟ إجابة قصيرة عن سؤال الأهل الطبيعي «ما فائدة هذا الرمز؟».

الأسطر 291–317

```
class _WhyRow extends StatelessWidget {
  const _WhyRow({required this.icon, required this.text});

  final String icon;
  final String text;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 6),
      child: Row(
        spacing: 12,
        children: [
          IconTile(
            iconName: icon,
            iconColor: AppColors.teal,
            color: AppColors.tealTint,
            size: 34,
            iconSize: 18,
            radius: 10,
          ),
          Expanded(child: Text(text, style: AppTextStyles.body15)),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ `_WhyRow`: مربّع أيقونة صغير أخضر ونص بجانبه.

لماذا بهذه الطريقة؟ سطر واحد يُعاد ثلاث مرات.

الأسطر 319–334

```
/// بطاقة معلومة: أيقونة ملوّنة وسطر عنوان وسطر توضيح.
class _InfoRow extends StatelessWidget {
  const _InfoRow({
    required this.icon,
    required this.iconColor,
    required this.tint,
    required this.title,
    required this.caption,
  });

  final String icon;
  final Color iconColor;
  final Color tint;
  final String title;
  final String caption;
```

ماذا يفعل؟ `_InfoRow`: حقول الأيقونة ولونيها والعنوان والتوضيح.

لماذا بهذه الطريقة؟ سطرا «حالة الحفظ» و«مجموعة الصف» بنفس الشكل بلونين مختلفين.

الأسطر 335–364

```
  @override
  Widget build(BuildContext context) {
    return LedgeBox(
      radius: 18,
      padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
      child: Row(
        spacing: 12,
        children: [
          IconTile(
            iconName: icon,
            iconColor: iconColor,
            color: tint,
            size: 40,
            iconSize: 20,
            radius: 12,
          ),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(title, style: AppTextStyles.body15),
                Text(caption, style: AppTextStyles.caption13),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ `LedgeBox` بزوايا 18: مربّع الأيقونة 40، والعنوان والتوضيح بجانبه.

لماذا بهذه الطريقة؟ شكل موحّد لبطاقات المعلومات في التطبيق.